Reconocimiento de Patrones

Universidad de Sonora · Licenciatura en Ciencias de la Computación
Semestre 2026-2 · Prof. Julio Waissman Vilanova

Examen intermedio para llevar · Examen B

Unidades 1 a 5

Instrucciones

Plazo y forma. El examen es **individual** y tienes **una semana** a partir de su publicación en Teams. Tiene dos problemas de 50 puntos cada uno. Cada problema tiene una parte teórica, una parte práctica y una bitácora de decisiones.

Tus parámetros personales. Varias partes del examen dependen de tu número de expediente. Sean

$$a = \text{último dígito de tu expediente}, \qquad b = \text{penúltimo dígito de tu expediente},$$

y usa tu expediente completo como semilla de todo lo aleatorio. Por ejemplo, con el expediente 226201347 se tiene $a = 7$, $b = 4$ y la semilla es 226201347. Escribe a, b y la semilla en la primera página de tu PDF y en el `README` de tu repositorio. Si resuelves el examen con los parámetros de otra persona, el problema vale cero.

Qué entregas en Teams.

1. **Un PDF escrito en LaTeX** (puedes usar Overleaf o lo que prefieras) con la parte teórica y la bitácora de decisiones de los dos problemas. No se aceptan documentos escritos a mano ni en Word: parte del objetivo es que practiques la escritura matemática formal. Las demostraciones deben estar completas y redactadas en prosa, con cada paso justificado; las fórmulas sueltas no cuentan como demostración.

2. **La liga a un repositorio de GitHub** con la parte práctica. Mientras dure el examen el repositorio debe ser **privado**, con acceso para el profesor (el usuario se indica en Teams). La estructura es:

   ```
   examen-intermedio/
   |-- README.md          # nombre, expediente, a, b, cómo reproducir todo
   |-- pyproject.toml     # o requirements.txt con versiones
   |-- problema-1/        # código, resultados y README.md del problema 1
   `-- problema-2/        # código, resultados y README.md del problema 2
   ```

   Cada carpeta tiene su propio `README.md` con los comandos para reproducir los resultados, las tablas y gráficas principales y lo que concluyes de ellas. El código puede estar en libretas o en archivos `.py`, pero tiene que correr de principio a fin en un entorno limpio. Los datos se descargan de su fuente original desde el propio código (cada problema dice de dónde); no los subas al repositorio. Se califica el último *commit* antes del cierre.

Decisiones. Los recuadros marcados con **Tú decides** no tienen una respuesta única. Se califica que la decisión sea razonable, que la apoyes con evidencia (una derivación, una gráfica o un experimento) y que entiendas qué pierdes con ella.

Bitácora de decisiones. Al final de cada problema, en el PDF, escribe una bitácora con al menos tres decisiones importantes que tomaste. Para cada una: qué decidiste, qué alternativa descartaste, por qué, y qué evidencia te convenció. La bitácora es la base de la defensa oral.

Uso de asistentes de IA. Puedes usarlos, y debes declarar en el PDF para qué. Pero toma en cuenta que el examen depende de tus parámetros y de tus decisiones, así que tendrás que plantear tú las preguntas con precisión.

Defensa oral. Después de la entrega tendrás una cita breve e individual. Te voy a pedir que expliques tus demostraciones, tu código y tu bitácora. **Si no puedes explicar lo que entregaste, asumiré que lo hizo una IA generativa y que no lo estudiaste ni intentaste entenderlo, y ese problema vale cero.**

Descuentos generales. Código que no corre de principio a fin: -5 por problema. Tablas o gráficas sin un comentario que diga qué significan: hasta -3 por problema. Gráficas sin ejes con nombre: -1 cada una.

1. El reporte demasiado bueno. **(50 puntos)**

Un banco portugués contrató a la consultora «Predicciones Certeras» para predecir qué clientes van a contratar un depósito a plazo después de una llamada de su campaña de mercadeo. La consultora entregó un modelo con una exactitud de 93 % y un AUC de 0.98, y el código de abajo como prueba. Al banco le parece demasiado bueno para ser cierto y te pide una segunda opinión.

Los datos son *Bank Marketing* del *UCI Machine Learning Repository*: 45,211 llamadas hechas entre mayo de 2008 y noviembre de 2010, en orden cronológico, con datos del cliente, de la llamada y si contrató o no (`y`). Lee la descripción de las variables en `https://archive.ics.uci.edu/dataset/222/bank+marketing`.

```
import io, zipfile, urllib.request
import numpy as np, pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, roc_auc_score

URL = "https://archive.ics.uci.edu/static/public/222/bank+marketing.zip"
externo = zipfile.ZipFile(io.BytesIO(urllib.request.urlopen(URL).read()))
interno = zipfile.ZipFile(io.BytesIO(externo.read("bank.zip")))
datos = pd.read_csv(interno.open("bank-full.csv"), sep=";")

y = (datos.pop("y") == "yes").astype(int)
X = pd.get_dummies(datos, drop_first=True).astype(float)

# Las clases están desbalanceadas: repetimos los positivos para equilibrarlas
pos = y == 1
X = pd.concat([X] + [X[pos]] * 6, ignore_index=True)
y = pd.concat([y] + [y[pos]] * 6, ignore_index=True)

X = StandardScaler().fit_transform(X)
X = SelectKBest(f_classif, k=15).fit_transform(X, y)

X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2, random_state=0)

mejor = None
for prof in [4, 8, 16, None]:
    modelo = RandomForestClassifier(n_estimators=200, max_depth=prof,
                                    random_state=0, n_jobs=-1)
    modelo.fit(X_ent, y_ent)
    auc = roc_auc_score(y_pru, modelo.predict_proba(X_pru)[:, 1])
    if mejor is None or auc > mejor[0]:
        mejor = (auc, prof, accuracy_score(y_pru, modelo.predict(X_pru)))

print(f"Profundidad elegida: {mejor[1]}")
print(f"Exactitud en prueba: {mejor[2]:.3f}   AUC en prueba: {mejor[0]:.3f}")
```

Tu versión del reporte. Cambia `k=15` por $k = 10 + a$ y `random_state=0` por tu semilla en todos lados. Si b mód $3 = 1$, cambia el bosque por `HistGradientBoostingClassifier(max_depth=prof)`, con la misma rejilla de profundidades; si b mód $3 = 2$, por `LogisticRegression(C=c, max_iter=2000)` con $c \in \{0.01, 0.1, 1, 10\}$.

Propiedades que puedes usar sin demostrar.

(P1) Si $Z_1, \ldots, Z_m$ son normales estándar independientes y m es grande, $\mathbb{E}[\max_j Z_j] \approx \sqrt{2 \ln m}$.

(P2) Si un clasificador tiene exactitud real p, su exactitud medida en n datos de prueba independientes es aproximadamente normal, con media p y desviación estándar $\sqrt{p(1-p)/n}$.

(P3) Si x y y son independientes, la correlación muestral entre ellas con n datos es aproximadamente normal, con media 0 y desviación estándar $1/\sqrt{n}$.

Parte teórica **(25 puntos)**

(a) **Copias en la prueba.** En los datos hay 39,922 llamadas sin contratación y 5,289 con contratación. Después de repetir los positivos como en el código:

1. ¿Qué fracción de los renglones son positivos?
2. Al partir al azar 80/20, ¿cuál es la probabilidad de que un renglón positivo de la prueba tenga al menos una copia idéntica en el entrenamiento? (Supón, para simplificar, que cada copia cae en entrenamiento o prueba de forma independiente.)
3. Si el modelo acierta siempre en los renglones de prueba que tienen una copia en el entrenamiento, y en los demás tiene exactitud p, ¿qué exactitud mide la consultora? Escríbela como función de p.

(6)

(b) **Elegir con la prueba.** La consultora probó varios modelos y reportó el mejor resultado *en la prueba*. Supón que se prueban m modelos independientes, todos con la misma exactitud real p. Usa (P1) y (P2) para mostrar que la exactitud del mejor, medida en la prueba, sobreestima p en aproximadamente $\sqrt{p(1-p)/n}\,\sqrt{2 \ln m}$. Calcula ese sesgo para el caso de la consultora y para tu caso: $m = 10^{1+(a \bmod 3)}$ modelos y $n = 100(b+1)$ datos de prueba, con $p = 0.8$. ¿Cuándo importa este problema y cuándo no? (5)

(c) **Seleccionar atributos con ruido.** Supón $n = 50$ datos y 5,000 atributos que son puro ruido, independientes de la clase. Usa (P1) y (P3) para estimar la mayor correlación (en valor absoluto) entre un atributo y la clase. Explica por qué, si se eligen los 20 atributos más correlacionados con *todos* los datos y después se hace validación cruzada, la exactitud estimada puede ser mucho mayor que 50 %. ¿Cuál es la forma correcta de hacerlo? (6)

(d) **¿Es trampa la duración?** La variable `duration` es la duración de la llamada en segundos.

Tú decides. Da una definición precisa de *fuga de información* que dependa del momento en que se usa el modelo. Con ella decide si `duration` es una fuga. ¿Hay algún uso del modelo para el que no lo sea? ¿Alguna otra variable tiene el mismo problema?

(4)

(e) **El tiempo.** Los datos están en orden cronológico. Explica por qué una partición al azar puede dar una estimación demasiado optimista (o demasiado pesimista) del desempeño que tendrá el modelo en la próxima campaña. Propón una partición que imite cómo se va a usar. (4)

Parte práctica **(20 puntos)**

Trabaja en la carpeta `problema-1/` de tu repositorio.

(a) **Reproducir.** Corre tu versión del código de la consultora y reporta sus números.

(b) **Encontrar los problemas.** Haz una tabla con *todos* los problemas que encuentres en el código: qué es, de qué tipo es (fuga de información, mala práctica de validación, otra cosa) y por qué infla el resultado. Hay más de tres.

(c) **Medir cada uno.** Corrige los problemas *uno a la vez* (dejando los demás como están) y después todos juntos. Presenta una tabla con la exactitud y el AUC de cada variante. ¿Cuál pesa más? ¿Hay alguno que casi no cambie el resultado? ¿Por qué? Compara el efecto de las copias con tu fórmula del inciso (a).

(d) **La versión honesta.** Construye un modelo sin esos problemas: todo el preprocesamiento dentro de un `Pipeline`, el desbalance tratado solo con el entrenamiento, los hiperparámetros elegidos con validación cruzada en el entrenamiento y una sola evaluación en la prueba. Reporta el resultado con un intervalo de confianza (por ejemplo, con *bootstrap* de la prueba).

> **Tú decides.** Cómo partir los datos (al azar o respetando el tiempo), cómo tratar el desbalance y con qué métrica reportar, pensando en cómo usaría el banco el modelo: llamar a los clientes con mayor probabilidad de contratar.

(e) **El experimento del ruido.** Comprueba el inciso (c) con simulación. Con $n = 50$, 5,000 atributos de ruido, $20 + 5b$ atributos seleccionados y un clasificador que tú eliges, repite al menos 50 veces la validación cruzada con la selección hecha (i) antes y (ii) dentro de cada partición. Grafica las dos distribuciones de la exactitud estimada.

(f) **Carta al banco.** En el PDF, en no más de una página y sin jerga técnica, escribe la respuesta al banco: qué estaba mal, qué desempeño es realista y qué recomiendas.

Bitácora de decisiones **(5 puntos)**

Al menos tres decisiones de este problema, como se explica en las instrucciones. Incluye las de los recuadros **Tú decides**.

Cómo se califica

Parte	Puntos	Qué se espera
Teoría (a)–(c)	17	Cálculos correctos con las propiedades y explicación de lo que significa cada resultado.
Teoría (d)–(e)	8	Definición de fuga precisa y aplicada con criterio; partición que imita el uso.
Práctica	20	Reproducción; tabla completa de problemas; efecto de cada uno medido y explicado; versión honesta con intervalo; simulación del ruido; carta clara.
Bitácora	5	Decisiones con alternativas, razones y evidencia.

2. Árboles que cortan en diagonal. (50 puntos)

Un árbol de decisión pregunta una variable a la vez: «¿$x_3 \leq 2.7$?». Por eso sus fronteras están hechas de segmentos paralelos a los ejes. Si la frontera real es una diagonal, como $x_1 > x_2$, el árbol tiene que aproximarla con una escalera de muchos escalones. Un *árbol oblicuo* pregunta en cada nodo por una combinación lineal de las variables: «¿$\mathbf{w}^\top \mathbf{x} \leq t$?». Así, una diagonal se corta de un solo golpe.

El problema es elegir $\mathbf{w}$. Una idea es usar en cada nodo un modelo lineal de la Unidad 4: la dirección del análisis discriminante lineal (LDA) con los datos que llegan a ese nodo. En este problema vas a construir ese árbol y averiguar cuándo vale la pena.

Propiedades que puedes usar sin demostrar.

(P1) *Dirección de Fisher.* Si en un nodo las clases 0 y 1 tienen medias $\mathbf{m}_0$ y $\mathbf{m}_1$, la dirección que mejor las separa en el sentido de LDA es $\mathbf{w} = S_W^{-1}(\mathbf{m}_1 - \mathbf{m}_0)$, donde $S_W = \sum_{i \in \text{clase } 0}(\mathbf{x}_i - \mathbf{m}_0)(\mathbf{x}_i - \mathbf{m}_0)^\top + \sum_{i \in \text{clase } 1}(\mathbf{x}_i - \mathbf{m}_1)(\mathbf{x}_i - \mathbf{m}_1)^\top$.

(P2) *Cuántas maneras hay de cortar.* Si n puntos de $\mathbb{R}^d$ están en posición general, el número de maneras distintas de separarlos en dos grupos con un hiperplano es $2\sum_{i=0}^{d}\binom{n-1}{i}$.

Parte teórica (25 puntos)

(a) **La escalera.** Sean los datos uniformes en el cuadrado $[0,1]^2$ con clase 1 si $x_2 > x_1$. Un árbol con cortes paralelos a los ejes puede aproximar la diagonal con una escalera de m escalones iguales. Calcula el área mal clasificada por esa escalera y muestra que es $1/(2m)$. ¿Cuántas hojas necesita, aproximadamente, para equivocarse en menos del 1 %? ¿Y un árbol oblicuo? (5)

(b) **Un nodo a mano.** En un nodo llegan estos seis puntos:

$$\text{clase 0: } (0,1),\ (1,3),\ (3,4); \qquad \text{clase 1: } (1,0),\ (3,1),\ (4,3).$$

Muestra que ningún corte paralelo a un eje los separa sin errores. Calcula a mano $\mathbf{m}_0$, $\mathbf{m}_1$, S_W y la dirección de Fisher (P1); proyecta los seis puntos sobre ella y encuentra un umbral que los separe. Haz un dibujo con los puntos y los dos cortes. (5)

(c) **Cuántos cortes hay que revisar.** Para un corte paralelo a un eje basta revisar $n-1$ umbrales por variable. Con (P2), compara cuántos cortes distintos hay de cada tipo para $n = 1{,}000$ puntos en $d = 4$ y en $d = 30$ dimensiones. ¿Se pueden revisar todos los cortes oblicuos? ¿Por qué un corte más flexible puede sobreajustar más, en términos de sesgo y varianza? (5)

(d) **Qué no les afecta a cada uno.**

1. Muestra que los cortes de un árbol con cortes paralelos a los ejes no cambian (salvo por el valor del umbral) si a una variable se le aplica una función estrictamente creciente, como log.
2. Muestra que, si todos los datos se transforman como $\mathbf{x}' = A\mathbf{x} + \mathbf{c}$, con A invertible, la dirección de Fisher se transforma como $\mathbf{w}' = A^{-\top}\mathbf{w}$ y que $\mathbf{w}'^\top \mathbf{x}' = \mathbf{w}^\top \mathbf{x} + \text{constante}$. Concluye que los cortes del árbol oblicuo no cambian si se giran, se estiran o se trasladan los datos.
3. ¿Es el árbol con cortes paralelos invariante a una rotación? ¿Es el árbol oblicuo invariante a un logaritmo?

(6)

(e) **Tu diseño.** Escribe en pseudocódigo tu algoritmo completo: cómo calcular $\mathbf{w}$ en cada nodo, cómo elegir el umbral, cuándo detenerse y qué predice cada hoja.

> **Tú decides.** Qué hacer cuando S_W no es invertible (por ejemplo, en un nodo con pocos datos), y si en cada nodo usas siempre el corte oblicuo o comparas contra el mejor corte paralelo a un eje y te quedas con el mejor.

(4)

Parte práctica (20 puntos)

Trabaja en la carpeta `problema-2/` de tu repositorio. Tus datos reales dependen de a mód 4 y los descarga tu propio código del *UCI Machine Learning Repository* (lee los archivos de descripción que vienen dentro):

a mód 4	Datos	Fuente y clase
0	Billetes falsos	`https://archive.ics.uci.edu/static/public/267/banknote+authentication.zip`; la última columna
1	Cáncer de mama	`https://archive.ics.uci.edu/static/public/17/breast+cancer+wisconsin+diagnostic.zip`; maligno (`M`) o benigno
2	Correo basura	`https://archive.ics.uci.edu/static/public/94/spambase.zip`; la última columna
3	Vino tinto	`https://archive.ics.uci.edu/static/public/186/wine+quality.zip`; `winequality-red.csv`, clase 1 si la calidad es 7 o más

(a) **Tu árbol.** Programa tu árbol oblicuo con NumPy, para clasificación binaria, como una clase con los métodos `fit`, `predict` y `predict_proba` (hereda de `BaseEstimator` y `ClassifierMixin` de scikit-learn para poder usar `cross_val_score`). El umbral de cada nodo se elige con el índice de Gini sobre los datos proyectados. Incluye una prueba automática (`assert`) de que con los seis puntos del inciso (b) obtienes una dirección proporcional a la tuya y un nodo sin errores.

(b) **Girar y transformar.** Genera 1,000 puntos uniformes en $[-1, 1]^2$ con clase 1 si $x_1 > 0.2$ y gíralos un ángulo de $10 + 5b$ grados. Compara la exactitud (con validación cruzada) de tu árbol, del `DecisionTreeClassifier` de scikit-learn y de un bosque aleatorio, para profundidades de 1 a 8, con los datos sin girar y girados. Después, con los datos sin girar, aplica $x_1 \mapsto e^{3x_1}$ y repite. ¿Coincide con el inciso (d)? Si algún modelo no cambia cuando esperabas que cambiara, explica por qué.

(c) **Tus datos reales.** Compara con validación cruzada tu árbol, el árbol de scikit-learn, un bosque aleatorio y un *bagging* de tus árboles (`BaggingClassifier`), eligiendo la profundidad de cada uno. Reporta también el número de hojas y el tiempo de entrenamiento.

> **Tú decides.** La métrica de comparación (revisa qué tan desbalanceadas están las clases) y cómo eliges la profundidad.

(d) **Cuándo vale la pena.** Con tus resultados y los del inciso anterior, ¿en qué tipo de datos esperas que gane el árbol oblicuo y en cuáles que pierda? Si con tus datos perdió, propón una explicación y ponla a prueba con un experimento.

Bitácora de decisiones **(5 puntos)**

Al menos tres decisiones de este problema, como se explica en las instrucciones. Incluye las de los recuadros **Tú decides**.

Cómo se califica

Parte	Puntos	Qué se espera
Teoría (a)–(c)	15	Área de la escalera; nodo calculado a mano con dibujo; conteo de cortes y relación con sesgo y varianza.
Teoría (d)–(e)	10	Las dos invariancias demostradas; pseudocódigo completo con las decisiones justificadas.
Práctica	20	Árbol programado y probado; experimento de rotación y transformación; comparación con datos reales con métrica y profundidad bien elegidas; conclusión puesta a prueba.
Bitácora	5	Decisiones con alternativas, razones y evidencia.